MANIFOLD

Complete Substrate Architecture Design Document

Diamond Edition — Definitive Specification v1.0

Author: Ken Bingham

Organization: Ken's Games / ButterflyFX

Governing Equation: f = x · y

Source Path: C:\manifold\

License: Proprietary — Ken Bingham / Ken's Games

Date: April 12, 2026

Classification: Technical Specification & AI Directive

Executive Summary

This document defines the complete mathematical, architectural, and implementation specification for the **Manifold** substrate system — the foundational computational framework powering the **ButterflyFX Dimensional Programming Platform**. It is the single source of truth for all ButterflyFX-powered applications. Any AI agent, developer, or system consuming this document SHALL be able to implement a fully compliant substrate constellation from scratch.

The Manifold architecture replaces traditional event-driven, callback-based software design with a purely geometric paradigm: **substrates publish and read coordinates on shared mathematical surfaces**. There are no function calls between modules, no callbacks, no event buses, and no message queues. Communication happens through the topology of the manifold itself.

The governing equation **z = x · y** — a hyperbolic paraboloid (saddle surface) — serves as the primary computational manifold upon which game state, audio synthesis, AI decisions, and physics all evaluate. The golden ratio **φ = 1.6180339887...** governs all proportional relationships. The Fibonacci sequence

governs all weighting distributions. The Schwartz Diamond surface provides the topological template for substrate interconnection.

> *"Dimensions ARE the computation. Substrates do not call each other — they publish coordinates on shared manifold surfaces. Other substrates read those surfaces. No callbacks. No spaghetti. Just geometry."*
>
> — Ken Bingham, ButterflyFX Design Philosophy

This specification governs all projects in the Ken's Games ecosystem, including **Fast Track**, **Starfighter**, and all future titles. Every value herein is explicit and deterministic. No approximations. No magic numbers. Every dimension cascades from φ.

Table of Contents

Section 1 — Project Identity

Section 2 — Mathematical Foundations

- 2.1 Primary Manifold Surface: $z = x \cdot y$
- 2.2 Secondary Manifold Surface: $z = x \cdot y^2$
- 2.3 The Golden Ratio: φ
- 2.4 The Fibonacci Sequence
- 2.5 Musical Frequency Formula
- 2.6 The Schwartz Surface Connection
- 2.7 Three States of Every Point

Section 3 — The ButterflyFX 7-Layer Dimensional Helix

Section 4 — Substrate Architecture: The Diamond Pattern

- 4.1 What Is a Substrate?
- 4.2 The Diamond Communication Pattern
- 4.3 Substrate Lifecycle
- 4.4 Surface Registry

4.5 Standard Substrate Catalog

Section 5 — Implementation Specification

5.1 Language and Runtime

5.2 File Structure Convention

5.3 Substrate Class Template

5.4 Main Loop Structure

5.5 Surface Registry Implementation

5.6 Surface Evaluation Implementation

Section 6 — Application to Fast Track

Section 7 — Application to Starfighter

Section 8 — The Diamond Topology in Detail

Section 9 — Extending the Manifold

Section 10 — Quality Standards and Rendering Mandates

Section 11 — Commercial Deployment

Section 12 — Glossary of Manifold Terms

Section 1 — Project Identity

Field	Value
Project Name	Manifold — Diamond Substrate Architecture
Author	Ken Bingham
Organization	Ken's Games / ButterflyFX
Governing Equation	$f = x \cdot y$
Design Philosophy	"Dimensions ARE the computation."
Source Path	C:\manifold\
License	Proprietary — Ken Bingham / Ken's Games

Field	Value
Edition	Diamond Edition v1.0
Applications Governed	Fast Track, Starfighter, and all future Ken's Games titles
Document Classification	Technical Specification & AI Directive

Purpose: This document defines the complete mathematical, architectural, and implementation specification for the Manifold substrate system. It is the single source of truth for all ButterflyFX-powered applications. Any AI agent, developer, or system consuming this document SHALL be able to implement a fully compliant substrate constellation from scratch.

DIRECTIVE

Substrates do not call each other — they publish coordinates on shared manifold surfaces. Other substrates read those surfaces. No callbacks. No spaghetti. Just geometry. Every value in this document is explicit and deterministic. There are no "approximately" values except for irrational constants (φ, π, √5). All dimensions, proportions, and relationships cascade mathematically from the governing equation and the golden ratio.

Section 2 — Mathematical Foundations

The Manifold system rests upon a small number of mathematical primitives from which all complexity emerges. These foundations are not metaphors — they are literal computational surfaces, sequences, and ratios that govern every value in every application built on ButterflyFX.

2.1 The Primary Manifold Surface: z = x · y

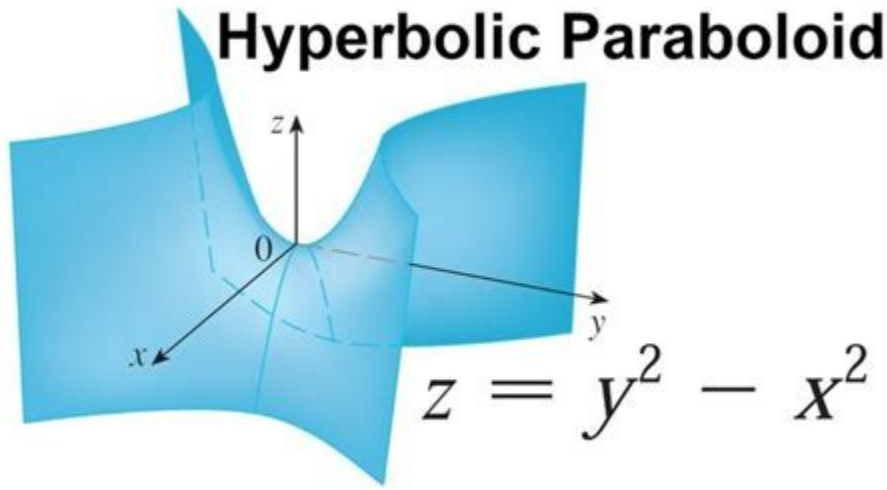

Figure 2.1 — The Relation surface z = x·y: a hyperbolic paraboloid (saddle surface) in 3D space.

Property	Value
Name	The "Relation" Surface
Equation	z = x · y
Geometry	Hyperbolic paraboloid (saddle surface) in 3D space
Origin	The surface passes through the origin (0, 0, 0)
Along y = x	$z = x^2$ — the surface rises parabolically
Along y = −x	$z = -x^2$ — the surface descends parabolically
Saddle Point	At the origin — a critical point with zero gradient
Gradient	$\nabla f = (y, x)$
Gradient Magnitude	$\lvert\nabla f\rvert = \sqrt{(x^2 + y^2)}$

Role in the System: This surface is the primary computational manifold. Game state, audio synthesis, AI decisions, and physics all evaluate coordinates on this surface. The z-value at any (x, y) coordinate encodes information — pitch, volume, position, score, probability, or any domain-specific quantity. Every point (x, y) maps to exactly one unique z-value, making the system deterministic and reproducible.

Gradient Interpretation: The gradient $\nabla f = (y, x)$ at any point tells you the direction of steepest ascent on the surface. AI agents walk along these gradients to make decisions. The gradient's magnitude $|\nabla f| = \sqrt{(x^2 + y^2)}$ indicates decision confidence — higher magnitude means a steeper surface and therefore a more decisive evaluation.

2.2 The Secondary Manifold Surface: $z = x \cdot y^2$

Property	Value
Name	The "Form" Surface
Equation	$z = x \cdot y^2$
Geometry	Curved bowl/trough — asymmetric, with steeper rise in the y direction
Gradient	$\nabla f = (y^2, 2xy)$
Non-linearity	Quadratic in y — creates richer harmonic content and more complex decision landscapes

Role in the System: The Form surface is used for sound effect synthesis, harmonic overtone generation, and advanced AI evaluation where non-linear response is needed. Its asymmetry in the y dimension produces richer harmonic content when mapped to audio parameters, and more nuanced decision landscapes when used for AI evaluation. The quadratic y-dependence creates natural crescendo and diminuendo curves for audio envelopes.

2.3 The Golden Ratio: φ = (1 + √5) / 2

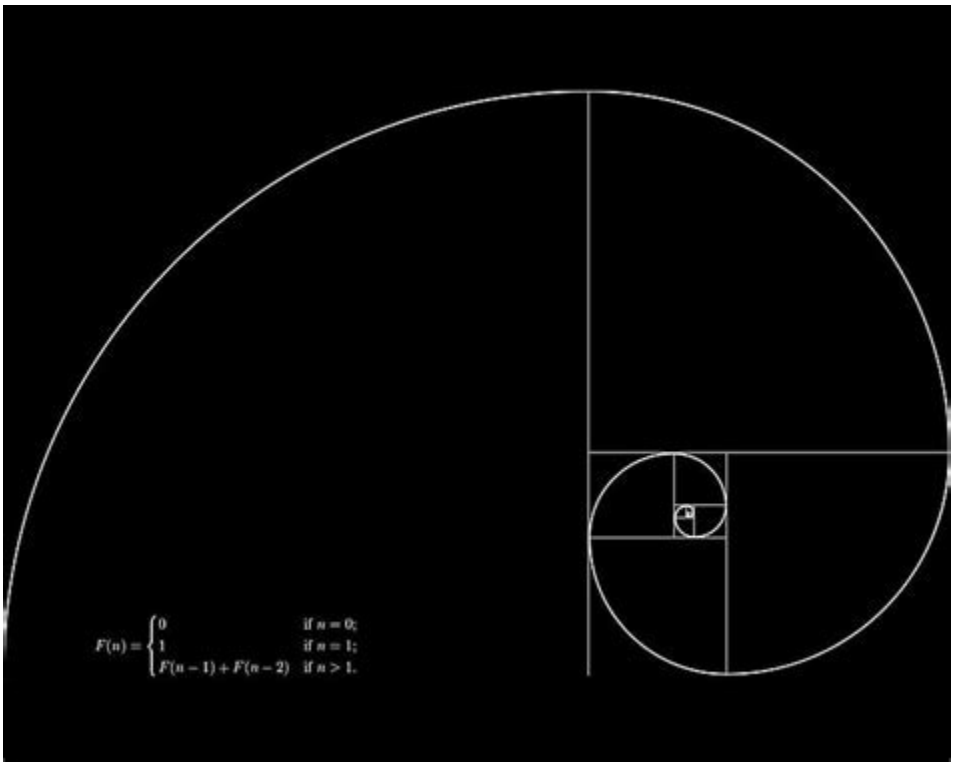

Figure 2.3 — The Fibonacci spiral converging to the golden ratio φ — the universal proportion governing all Manifold dimensions.

The golden ratio **φ = 1.6180339887498948482...** governs ALL proportions in the Manifold system. No arbitrary pixel values. No magic numbers. All dimensions are φ-derived.

Constant	Value	Usage
φ	1.6180339887...	Primary proportion ratio for all spatial and temporal relationships
φ^2	2.6180339887... (= φ + 1)	Self-similar scaling — used for recursive layout subdivision
1/φ	0.6180339887... (= φ − 1)	Reciprocal relationship — used for inverse proportions and timing decay
φ^3	4.2360679774...	Third-order scaling for fine detail elements
φ^4	6.8541019662...	Fourth-order scaling for

Constant	Value	Usage
		ultra-fine detail (borders, strokes)
Golden Angle	$2\pi / \varphi^2 \approx 137.5077640500...$	Radial element distribution — phyllotactic spacing for circular layouts

Cascade Principle: Every layout decision cascades from φ. Board dimensions, hole spacing, peg sizes, UI element ratios, animation timing curves, and camera field-of-view ratios are all computed as multiples or divisions of φ applied to a base unit R. This guarantees visual harmony across every element at every scale.

2.4 The Fibonacci Sequence

The Fibonacci sequence is the discrete integer manifestation of the golden ratio. Its consecutive ratios converge to φ, providing a natural bridge between integer-domain operations (frame counts, layer indices, scoring tiers) and continuous-domain proportions.

Index (n)	1	2	3	4	5	6	7	8	9	10	11	12
F(n)	1	1	2	3	5	8	13	21	34	55	89	144

- **Recurrence:** $F(n) = F(n-1) + F(n-2)$, with $F(1) = F(2) = 1$
- **Convergence:** $\lim(n\to\infty) F(n+1)/F(n) = \varphi$
- **Audio layer weighting formula:** weight(layer_i) = F(i) / Σ F(all_layers)
- **Animation frame counts:** Transitions use Fibonacci frame counts (5, 8, 13, 21 frames)
- **Timing intervals:** Delay sequences follow Fibonacci millisecond values
- **Difficulty curve progression:** AI difficulty scales along Fibonacci tiers
- **Scoring multipliers:** Bonus multipliers drawn from Fibonacci values

2.5 Musical Frequency Formula

A single equation generates every note on the chromatic scale:

FREQUENCY FORMULA

$$f = 440 \times 2^{((n - 69) / 12)}$$

Where **n** is the MIDI note number. Middle A (A4) = 69. Middle C (C4) = 60.

Note	MIDI Number (n)	Frequency (Hz)
C4 (Middle C)	60	261.6256
D4	62	293.6648
E4	64	329.6276
F4	65	349.2282
G4	67	391.9954
A4 (Concert A)	69	440.0000
B4	71	493.8833
C5	72	523.2511

Manifold Integration: The z-value at a surface coordinate maps to n (the MIDI note number), which maps to frequency via the formula above. This allows the Music Substrate to derive pitch directly from game state coordinates on the Relation surface. The **octave relationship** is: doubling frequency = +12 semitones = one octave up.

2.6 The Schwartz Surface Connection

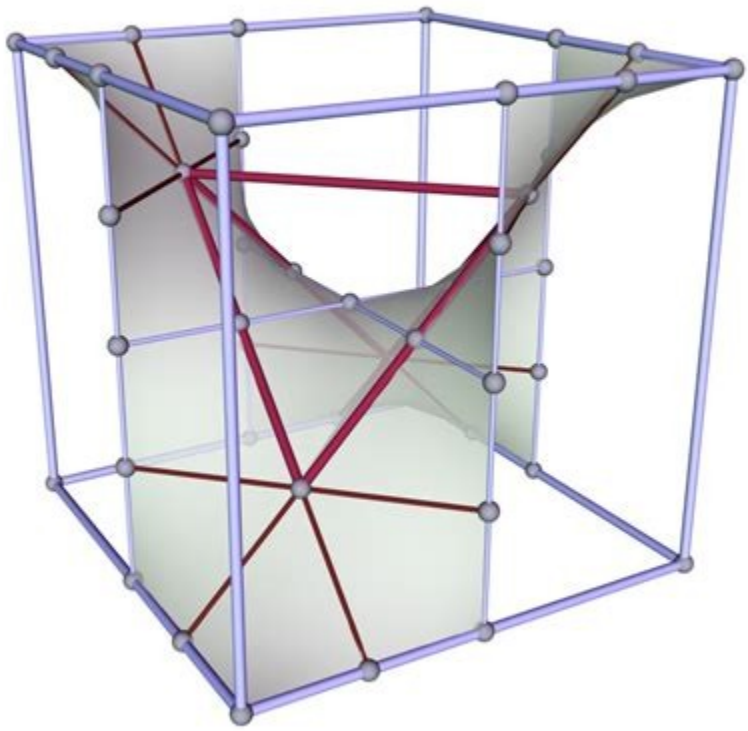

Figure 2.6 — The Schwartz D-Surface (Diamond surface): a triply periodic minimal surface whose topology provides the template for substrate interconnection.

The Schwartz minimal surfaces (Schwartz P-surface and D-surface) provide the topological template for substrate interconnection within the Manifold architecture.

Property	Description
Surface Type	Schwartz D-surface (Diamond surface) — triply periodic minimal surface
Mean Curvature	$H = 0$ everywhere (minimal surface — every point is a saddle point)
Gaussian Curvature	$K \leq 0$ everywhere (saddle-like at every point)
Symmetry Group	$Fd\bar{3}m$ (diamond cubic)
Topology	Divides R^3 into two congruent, interlocking labyrinths

Architectural Mapping: The two interlocking labyrinthine channels of the Diamond surface represent:

1. **Channel A — The Data/State Channel:** Coordinates published by substrates (write path)
2. **Channel B — The Observation/Read Channel:** Coordinates consumed by substrates (read path)

This enforces the fundamental principle: **substrates never directly couple**. They share a surface, not a call stack. The Diamond surface's symmetry group ($Fd\bar{3}m$) maps to the substrate constellation's symmetry — every substrate has equivalent topological status. No substrate is "closer to" or "more privileged than" any other.

2.7 Three States of Every Point

Every point in the Manifold system exists in exactly one of three states at any given moment:

State	Definition	Analogy
Potential	Unaddressed possibility. The coordinate exists in mathematical space but has not been evaluated. It represents latent capability.	An empty cell in a spreadsheet that has never been visited
Null	Addressed but valueless. The coordinate has been visited/evaluated but carries no manifest value. It is pure location — a placeholder in the topology.	A cell that has been selected but contains no data
Manifest	Addressed and valued. The coordinate has been evaluated and carries a concrete value (a note, a position, a decision, a color). This is actualized state.	A cell containing a formula result or entered value

State Transitions:

- **Forward:** Potential → Null (addressing) → Manifest (valuation)
- **Reverse:** Manifest → Null (clearing) → Potential (releasing)

The total state space of the system is the product of all coordinates across all substrates. At any frame, each coordinate is in exactly one of these three states, and the distribution of states across the constellation defines the system's overall configuration.

Section 3 — The ButterflyFX 7-Layer Dimensional Helix

ButterflyFX Dimensional Programming replaces traditional iteration and event-driven programming with **dimensional transitions**. Dimensions are not stacked layers in a conventional sense — **each dimension IS an entire lower dimension, carried within it**. Layer 3 does not sit "on top of" Layer 2; Layer 3 contains Layer 2 as an intrinsic aspect of its being.

THE SEVEN GENESIS LAYERS

Each layer is named for its role in the creation of computational reality. The helix structure means that upon reaching Layer 7 (Completion), the system spirals upward — Layer 7 becomes the Layer 1 Spark of the next dimensional spiral. This is the mechanism of infinite scalability.

Layer 1 — SPARK (The First Light)

Property	Value
Mathematical Analog	The point (0-dimensional)
Role	The initial seed. A single value. The

Property	Value
	origin coordinate. The "1" from which everything unfolds.
In Code	A single constant, a configuration seed, the entry point of computation.

Spark is the irreducible beginning — a single datum with no extent, no direction, no relation. It is the first act of addressing: a point transitions from Potential to Null to Manifest, and computation begins.

Layer 2 — MIRROR (Reflection)

Property	Value
Mathematical Analog	Duality (1-dimensional — the line between two points)
Role	Every value has its complement. Input/output. Read/write. State/observation.
In Code	Every substrate has a publish side and a read side. Every coordinate has a value and its conjugate.

Mirror introduces duality — the recognition that every quantity implies its opposite. The Schwartz D-surface's two interlocking channels emerge at this layer. The Mirror dimension establishes the fundamental read/write separation that makes the Diamond pattern possible.

Layer 3 — RELATION (Connection)

Property	Value
Mathematical Analog	The manifold surface $z = x \cdot y$ (2-dimensional surface in 3D space)
Role	The first true manifold. Two independent variables produce a dependent value.

Property	Value
In Code	The core evaluation function. Given (x, y), compute z.

Relation is WHERE computation happens — on the surface. This is the layer that gives the Manifold system its name. All game state, audio synthesis, AI evaluation, and physics simulation reduce to surface evaluation at this dimensional layer. The Relation surface $z = x \cdot y$ is the primary expression of this layer.

Layer 4 — FORM (Structure)

Property	Value
Mathematical Analog	The manifold surface $z = x \cdot y^2$ and higher-order surfaces (structured 2-manifolds with curvature)
Role	Structure emerges from relation. Non-linearity, harmonics, and complexity appear.
In Code	Structured data types, component hierarchies, scene graphs — all derived from surface topology.

Form adds curvature and asymmetry. Board layouts, UI hierarchies, entity hierarchies, and component trees all live at this dimensional layer. The Form surface $z = x \cdot y^2$ introduces the non-linear response that creates harmonic richness in audio and strategic depth in AI evaluation.

Layer 5 — LIFE (Animation)

Property	Value
Mathematical Analog	Time-parameterized curves on manifold surfaces (paths, trajectories)
Role	Points begin to move along the surface. Animation, physics, music playback — all are trajectories on manifolds.

Property	Value
In Code	Update loops, animation controllers, physics integrators — all evaluate surface coordinates at time t.

Life introduces time as a parameter. A peg moving around the board is a parameterized curve on the board manifold. A musical note's envelope is a trajectory on the Form surface. A ship's flight path is a curve on the velocity manifold. Everything that moves, changes, or evolves lives at this dimensional layer.

Layer 6 — MIND (Coherence)

Property	Value
Mathematical Analog	Gradient fields and optimization on manifold surfaces
Role	The system becomes aware of its own state. AI agents walk along gradients. Commentary observes and narrates.
In Code	AI evaluation functions, pattern recognition, strategic planning — all gradient-based.

Mind is coherence — the ability to evaluate and respond to the surface's shape. At this layer, the system is no longer merely following trajectories; it is analyzing the surface topology to make optimal decisions. The gradient $\nabla f = (y, x)$ becomes a decision compass. AI difficulty is a function of how faithfully the agent follows the true gradient versus a noisy approximation.

Layer 7 — COMPLETION (The Whole / The Butterfly)

Property	Value
Mathematical Analog	The full manifold with all layers carrying

Property	Value
	all lower dimensions
Role	The complete system. Every substrate, every surface, every coordinate, every trajectory — unified.
In Code	The running application. All substrates publishing and reading. The game playing. The music sounding. The AI thinking.

Completion is the "butterfly" — the system achieves self-similar wholeness. φ spirals to the next level. When Layer 7 completes, it becomes the Layer 1 Spark of the next dimensional spiral. This is how the system scales: each complete application becomes a seed for the next.

> **∞ → NEXT SPIRAL**
>
> Layer 7 → Layer 1 (next spiral). A completed Fast Track application becomes the Spark seed for the next application in the Ken's Games constellation. The substrate architecture, the manifold surfaces, the golden ratio cascades — all carry forward as the foundation upon which the next creation is built. The butterfly emerges, and its wings become the seed of the next butterfly.

Section 4 — Substrate Architecture: The Diamond Pattern

4.1 What Is a Substrate?

A substrate is a self-contained computational module that:

- **Owns a single domain** (audio, board state, rules, AI, etc.)
- **Evaluates coordinates** on shared manifold surfaces
- **Publishes its outputs** as coordinates on those surfaces
- **Reads inputs** by sampling coordinates published by other substrates
- **NEVER directly calls** another substrate's functions
- **NEVER passes callbacks, events, or messages** to other substrates
- **Has no knowledge** of which other substrates exist

A substrate is not a "service," a "component," or a "module" in any traditional software sense. It is a geometric entity that occupies a region of the manifold and communicates exclusively through the surface topology. Its isolation is absolute — if every other substrate were removed from the system, a substrate would still run without error (it would simply read Null values from unoccupied coordinates).

4.2 The Diamond Communication Pattern

Named after the Schwartz D-surface (Diamond surface), this pattern enforces total decoupling between substrates. The manifold surface IS the communication medium. There is no bus, no event system, no message queue.

```
Substrate A                Manifold Surface                Substrate B    │
│                          │     ├─── publishes ──────────────────────→ │
←──────────────────────── reads ──────────────┤     │                    │
│     ├─── reads ←──────────────────────────── │ ──────────────────────→
publishes ──────────┤     │                    │                         │     │   (no
direct connection     │   (the surface IS the medium)    │     │   between A and B)
│                          │
```

Substrates write coordinates. Other substrates read coordinates. The surface topology determines what information flows where. There is no dependency injection, no service locator, no observer pattern. There is only the surface.

4.3 Substrate Lifecycle

Every substrate follows the same lifecycle, without exception:

3. **INITIALIZE** — Set up internal state, register surface coordinates of interest. Happens once at startup.
4. **SAMPLE** — Read coordinates from shared surfaces (input phase).
5. **EVALUATE** — Process inputs through domain-specific logic (the substrate's own manifold function).
6. **PUBLISH** — Write output coordinates to shared surfaces.
7. **RENDER** — If the substrate has visual/audio output, produce it.

Steps 2 through 5 repeat on every frame/tick. The ordering is strict: ALL substrates sample before ANY substrate evaluates. ALL substrates evaluate before ANY substrate publishes. This guarantees frame consistency.

4.4 Surface Registry

The Surface Registry is the only shared infrastructure in the entire Manifold system. It is a flat dictionary mapping coordinate names to values. It has NO logic — it is pure data.

REGISTRY INVARIANTS

Single-Writer: Only one substrate MAY publish to a given coordinate. Violation SHALL throw an error at registration time.

Multi-Reader: Any number of substrates MAY read any coordinate.

No Dependency Graph: The registry does not know or care about read/write ordering.

Frame Consistency: All reads within a single frame see the same values (double-buffered). A substrate's writes in frame N are not visible to any reader until frame N+1.

4.5 Standard Substrate Catalog

The following table defines the standard substrate types that any ButterflyFX application MAY include. Not all applications require all substrates. Each substrate's domain, published coordinates, and read coordinates are defined below.

Substrate	Domain	Publishes	Reads
Board	Game board state	Hole positions, occupancy, connectivity	Theme coordinates, player state
Rules	Move validation	Legal moves, turn state, win condition	Board state, card state, peg positions
Peg	Piece state	Peg positions, animations, collisions	Board state, rules validation
CardDeck	Card management	Current card, hand contents, deck size	Rules state, player turn
Music	Procedural audio	Current beat, measure, tempo	Theme coordinates, game state
Audio	Sound effects	Active sounds, spatial positions	Game events (peg moves, cuts, wins)
AI	Computer opponents	Chosen move, difficulty state	Board state, legal moves, hand contents
Themes	Visual theming	Material parameters, colors, textures	User preference coordinates
Input	Player controls	Selected peg, selected card, camera state	Platform detection, device state
Social	Multiplayer/chat	Player profiles, messages, guild state	Auth state, network state
Lobby	Game rooms	Room state, player slots, game config	Social state, auth state
Auth	Authentication	Session state, user identity	Network state

Substrate	Domain	Publishes	Reads
Commentary	Play-by-play	Commentary text, speech synthesis	Game state, move history
Crowd	Ambient atmosphere	Crowd noise parameters, reaction intensity	Game state, score state
Avatar	Player representation	Avatar render data, customization	Auth state, social state
Render	Visual output	Frame buffer, post-processing	All visual substrate outputs

---

Section 5 — Implementation Specification

5.1 Language and Runtime

Layer	Technology	Notes
Primary Language	JavaScript (ES Modules, strict mode)	All application logic, substrate code, and orchestration
Shader Language	GLSL (vertex and fragment shaders)	All GPU-side rendering, post-processing, and visual effects
Graphics API	WebGL2	3D rendering, PBR materials, post-processing pipeline
Audio API	Web Audio API	Procedural synthesis, spatial audio, AudioWorklet DSP
Network API	WebSocket	Real-time multiplayer communication
XR API	WebXR	VR/AR support for Meta Quest and future

Layer	Technology	Notes
		headsets
Future Native	Electron, Capacitor, or platform-specific SDKs	Commercial release builds for Steam, mobile, and consoles

FRAMEWORK PROHIBITION

No frameworks. No jQuery. No React. No Angular. No Vue. No Svelte. No Next.js. No libraries of any kind except platform-native APIs. Pure vanilla JavaScript + WebGL2 + Web Audio. The Manifold IS the framework.

5.2 File Structure Convention

Every ButterflyFX project SHALL follow this exact directory structure:

```
project-root/ ├── index.html                    (entry point — single HTML
file) ├── js/ │   ├── manifold/ │   │   ├── surface.js            (z=xy,
z=xy² evaluation, gradient computation) │   │   ├── fibonacci.js
(sequence generation, weighting functions) │   │   ├── golden-ratio.js
(φ cascades, proportion derivation) │   │   ├── dimensions.js         (7-
layer helix state machine) │   │   └── registry.js           (Surface
Registry — coordinate read/write) │   ├── substrates/ │   │   ├──
board.substrate.js    (one file per substrate) │   │   ├── rules.substrate.js
│   │   ├── peg.substrate.js │   │   ├── cardDeck.substrate.js │   │   ├──
music.substrate.js │   │   ├── audio.substrate.js │   │   ├── ai.substrate.js
│   │   ├── themes.substrate.js │   │   ├── input.substrate.js │   │
└── ... │   ├── input/ │   │   ├── InputManager.js       (adapter pattern,
platform detection, hot-swap) │   │   ├── keyboard.adapter.js │   │   ├──
gamepad.adapter.js │   │   ├── touch.adapter.js │   │   └── xr.adapter.js │
├── render/ │   │   ├── board.renderer.js     (one file per visual domain) │
│   ├── peg.renderer.js │   │   ├── card.renderer.js │   │   └──
post.renderer.js │   └── main.js                   (bootstrap, substrate
initialization, main loop) ├── shaders/ │   ├── pbr.vert.glsl │   ├──
pbr.frag.glsl │   ├── post-bloom.frag.glsl │   ├── post-ao.frag.glsl │   └──
post-dof.frag.glsl ├── assets/ │   ├── models/ │   ├── textures/ │   └──
images/ └── docs/     ├── MANIFOLD_SPEC.md      └── GDD.md
```

5.3 Substrate Class Template

Every substrate MUST follow this exact class structure. No exceptions. No variations. The four lifecycle methods (`sample`, `evaluate`, `publish`, `render`) are the only interface contract.

```
// [name].substrate.js import { Registry } from '../manifold/registry.js';
import { evaluateSurface } from '../manifold/surface.js';  export class
NameSubstrate {   constructor(registry) {     this.registry =
registry;      // Declare published coordinates — this substrate is the SOLE
writer     this.registry.registerPublisher('domain.coordinate.name', this);
// Internal state only — no references to other substrates
this._inputValue = null;     this._outputValue = null;   }    sample()
{     // Read coordinates from the registry — NEVER call another substrate
const value = this.registry.read('other.coordinate.name');     // Store
locally for the evaluate phase     this._inputValue = value;   }
evaluate() {     // Domain-specific computation using manifold surface
evaluation     const z = evaluateSurface(this._inputValue, someOtherParam);
this._outputValue = z;   }    publish() {     // Write outputs to the
registry     this.registry.write('domain.coordinate.name',
this._outputValue);   }    render(ctx) {     // Optional: produce
visual/audio output     // Only substrates with user-facing output implement
this   } }
```

5.4 Main Loop Structure

The main loop is the heartbeat of the application. It orchestrates the four-phase substrate lifecycle across every frame. The phase ordering is inviolable.

```
// main.js — The heartbeat import { Registry } from './manifold/registry.js';
import { BoardSubstrate } from './substrates/board.substrate.js'; import
{ RulesSubstrate } from './substrates/rules.substrate.js'; import
{ PegSubstrate } from './substrates/peg.substrate.js'; import
{ CardDeckSubstrate } from './substrates/cardDeck.substrate.js'; import
{ MusicSubstrate } from './substrates/music.substrate.js'; import
{ AudioSubstrate } from './substrates/audio.substrate.js'; import
{ AISubstrate } from './substrates/ai.substrate.js'; import { ThemesSubstrate
} from './substrates/themes.substrate.js'; import { InputSubstrate } from
'./substrates/input.substrate.js';  const registry = new Registry();  const
substrates = [   new BoardSubstrate(registry),   new
RulesSubstrate(registry),   new PegSubstrate(registry),   new
CardDeckSubstrate(registry),   new MusicSubstrate(registry),   new
AudioSubstrate(registry),   new AISubstrate(registry),   new
ThemesSubstrate(registry),   new InputSubstrate(registry), ];  function
tick(timestamp) {   // Phase 1: SAMPLE — all substrates read from registry
for (const s of substrates) s.sample();    // Phase 2: EVALUATE — all
substrates compute   for (const s of substrates) s.evaluate();    // Phase 3:
PUBLISH — all substrates write to registry   for (const s of substrates)
s.publish();    // Phase 4: Registry double-buffer swap   registry.swap();
// Phase 5: RENDER — visual/audio substrates produce output   for (const s of
substrates) s.render(ctx);    requestAnimationFrame(tick); }
requestAnimationFrame(tick);
```

5.5 Surface Registry Implementation

The Registry is the only shared infrastructure. It is intentionally minimal — a double-buffered coordinate map with single-writer enforcement. It contains no logic, no routing, and no dependency tracking.

```
// registry.js export class Registry {   constructor() {     this._front = new Map();      // Current frame (read buffer)     this._back = new Map();      // Next frame (write buffer)     this._publishers = new Map(); // coordinate → substrate (single-writer enforcement)   }    registerPublisher(coordinate, substrate) {     if (this._publishers.has(coordinate)) {       throw new Error(         `Coordinate "${coordinate}" already has a publisher. Single-writer violated.`       );     }     this._publishers.set(coordinate, substrate);     this._front.set(coordinate, null);  // Initialize as Null state     this._back.set(coordinate, null);   }    read(coordinate) {     return this._front.get(coordinate);  // Always read from front buffer   }    write(coordinate, value) {     this._back.set(coordinate, value);   // Always write to back buffer   }    swap() {     // Swap buffers     [this._front, this._back] = [this._back, this._front];     // Copy front to back for next frame's baseline     for (const [key, val] of this._front) {       this._back.set(key, val);     }   } }
```

5.6 Surface Evaluation Implementation

These are the core mathematical functions that compute z-values and gradients on the Manifold's surfaces. Every substrate's `evaluate()` method ultimately calls one or more of these functions.

```
// surface.js export function evaluateRelation(x, y) {   return x * y;  // z = xy — the Relation surface }  export function evaluateForm(x, y) {   return x * y * y;  // z = xy² — the Form surface }  export function gradientRelation(x, y) {   return { dx: y, dy: x };  // ∇f = (y, x) }  export function gradientForm(x, y) {   return { dx: y * y, dy: 2 * x * y };  // ∇f = (y², 2xy) }  export function evaluateSurface(x, y, surfaceType = 'relation') {   switch (surfaceType) {     case 'relation': return evaluateRelation(x, y);     case 'form':     return evaluateForm(x, y);     default:         return evaluateRelation(x, y);   } }  export function gradientSurface(x, y, surfaceType = 'relation') {   switch (surfaceType) {     case 'relation': return gradientRelation(x, y);     case 'form':     return gradientForm(x, y);     default:         return gradientRelation(x, y);   } }  // Gradient magnitude — decision confidence metric export function gradientMagnitude(x, y, surfaceType = 'relation') {   const g = gradientSurface(x, y, surfaceType);   return Math.sqrt(g.dx * g.dx + g.dy * g.dy); }
```

Section 6 — Application to Fast Track

Fast Track is the flagship application of the Ken's Games ecosystem — a competitive board game featuring pegs, cards, and strategic movement on a hexagonal board. Every aspect of Fast Track is derived from the Manifold architecture.

6.1 Board Geometry from φ

All board dimensions cascade from a single base unit **R** (the board outer radius) through successive powers of the golden ratio:

Element	Formula	Derivation
Board outer radius	R	Base unit (1.0)
Hole diameter	R / φ^3	R / 4.2360679774...
Hole spacing (center-to-center)	R / φ^2	R / 2.6180339887...
Rail width	R / φ^4	R / 6.8541019662...
Safe zone length	$4 \times (R / \varphi^2)$	4 × R / 2.6180339887...
Rotational symmetry	6-fold (60°)	Hexagonal board — each player station identical under rotation

No arbitrary pixel values appear anywhere in the board layout. Every dimension is a mathematical consequence of R and φ.

6.2 Card Values as Surface Coordinates

Each card maps to an (x, y) coordinate on the Relation surface. The z-value at that coordinate determines the card's movement power:

Card	Coordinate (x, y)	z = x · y	Behavior
Ace	(1, 1)	1	Enter track + draw again
2	(2, 1)	2	Move 2 spaces forward
3	(3, 1)	3	Move 3 spaces forward
4	(4, −1)	−4	Move 4 spaces backward
5	(5, 1)	5	Move 5 spaces forward

Card	Coordinate (x, y)	z = x · y	Behavior
6	(6, 1)	6	Move 6 spaces forward
7	(7, 1)	7	Move 7 — splittable: (a, 1) + (b, 1) where a + b = 7
8	(8, 1)	8	Move 8 spaces forward
9	(9, 1)	9	Move 9 spaces forward
10	(10, 1)	10	Move 10 spaces forward
Face (J/Q/K)	(1, 1)	1	Move 1 + draw again + bull's eye exit
Joker	(1, 1)	1	Wild — enter track + draw again

6.3 AI Decision-Making via Gradient Descent

The AI evaluates every legal move by computing a "desirability" score on the Relation surface:

8. For each legal move, compute coordinates **(progress_toward_safe_zone, tactical_advantage)**
9. Evaluate **z = x · y** at those coordinates — the z-value is the move's desirability score
10. Select the move with the **highest z-value** (maximum ascent on the manifold)
11. For team play: evaluate moves for **both teammates' pegs** and select the globally optimal move

Difficulty Scaling:

- **Easy AI:** Adds Gaussian noise to gradient evaluation — the agent "stumbles" on the surface

- **Medium AI:** Follows gradients with occasional random perturbation
- **Expert AI:** Follows exact gradients with full look-ahead — optimal play

6.4 Music Generation from Manifolds

The Music Substrate reads the Board Manifold's current state coordinates and generates procedural music in real time:

Parameter	Surface Axis	Derived From
Music intensity (z)	$z = x \cdot y$	Product of game progress and game tension
Game progress (x)	x-axis	Average peg advancement across all players (0.0 to 1.0)
Game tension (y)	y-axis	Number of pegs at risk of being cut (0.0 to 1.0)

Fibonacci-Weighted Instrument Layers:

Layer	Instrument	Fibonacci Weight F(i)	Normalized Weight
Layer 1	Bass	$F(1) = 1$	$1 / \Sigma F$ (always active)
Layer 2	Drums	$F(2) = 1$	$1 / \Sigma F$
Layer 3	Melody	$F(3) = 2$	$2 / \Sigma F$
Layer 4	Harmony	$F(4) = 3$	$3 / \Sigma F$
Layer 5	Effects	$F(5) = 5$	$5 / \Sigma F$

Step Sequencer: 16 slots per bar, each slot triggers sounds from manifold coordinates. The sequencer reads the current z-value to determine which layers are active in the current bar.

Theme Songs: Each visual theme has a corresponding musical identity — same synthesis engine, different surface coordinates and tempos:

- **Neon Circuit** — Cosmic theme (synth-heavy, pulsing bass)

- **Stellar Drift** — Space Ace theme (ethereal pads, wide reverb)
- **Gladiator March** — Colosseum theme (brass-like overtones, martial rhythm)
- **Coral Groove** — Undersea theme (fluid modulation, bubble effects)

Section 7 — Application to Starfighter

Starfighter demonstrates the Manifold architecture's versatility — the same substrate system, the same surface evaluations, and the same golden ratio cascades that power a board game also power a real-time space combat game.

7.1 Flight Physics from Manifold Surfaces

Physics Parameter	Surface Mapping	Equation
Acceleration	Relation surface $z = x \cdot y$	z = thrust_input (x) × ship_mass_factor (y)
Velocity vector	Point on the Relation surface	Current velocity = surface coordinate
Drag	Negative gradient descent	Drag force pulls velocity toward the origin along $-\nabla f$
Terminal velocity	Equilibrium on the surface	Where thrust gradient equals drag gradient

7.2 Enemy AI via Gradient Fields

Each enemy type evaluates the Relation surface differently, producing distinct behavioral patterns from the same mathematical framework:

Enemy Type	Gradient Strategy	Behavior
Drone	Follows steepest gradient	Direct pursuit — simple,

Enemy Type	Gradient Strategy	Behavior
	toward player	relentless, predictable
Interceptor	Follows gradient perpendicular to player approach vector	Flanking maneuvers — cuts across the player's path
Bomber	Follows gradient toward baseship, ignoring player	Objective-focused — must be intercepted before reaching the baseship
Elite	Form surface $z = x \cdot y^2$ (non-linear pursuit curves)	Advanced maneuvering — unpredictable, asymmetric pursuit trajectories

7.3 Audio Synthesis

Sound Category	Surface	Coordinate Mapping
Weapon sounds	Form surface $z = x \cdot y^2$	Richer harmonics from quadratic y-dependence
Engine hum	Relation surface $z = x \cdot y$	(thrust_level, rpm) → low-frequency tone
Shield impacts	Form surface $z = x \cdot y^2$	Transient spikes — rapid attack, exponential decay
Ambient space	Relation surface $z = x \cdot y$	Low-amplitude, slowly varying coordinates

All synthesis uses the Web Audio API oscillators and AudioWorklet processors — **zero audio files**. Every sound in Starfighter is synthesized from manifold coordinates at runtime.

7.4 Radar System

- **Projection:** 3D sphere-to-2D disc projection uses manifold coordinate mapping — each 3D point maps to a 2D radar coordinate via surface evaluation

- **Distance attenuation:** Follows $1/\varphi^n$ curve (golden ratio falloff) — blips fade by the reciprocal golden ratio at each distance unit
- **Blip intensity:** z-value at (distance, threat_level) on the Relation surface — farther and less threatening enemies produce dimmer blips

---

Section 8 — The Diamond Topology in Detail

8.1 Schwartz D-Surface Properties

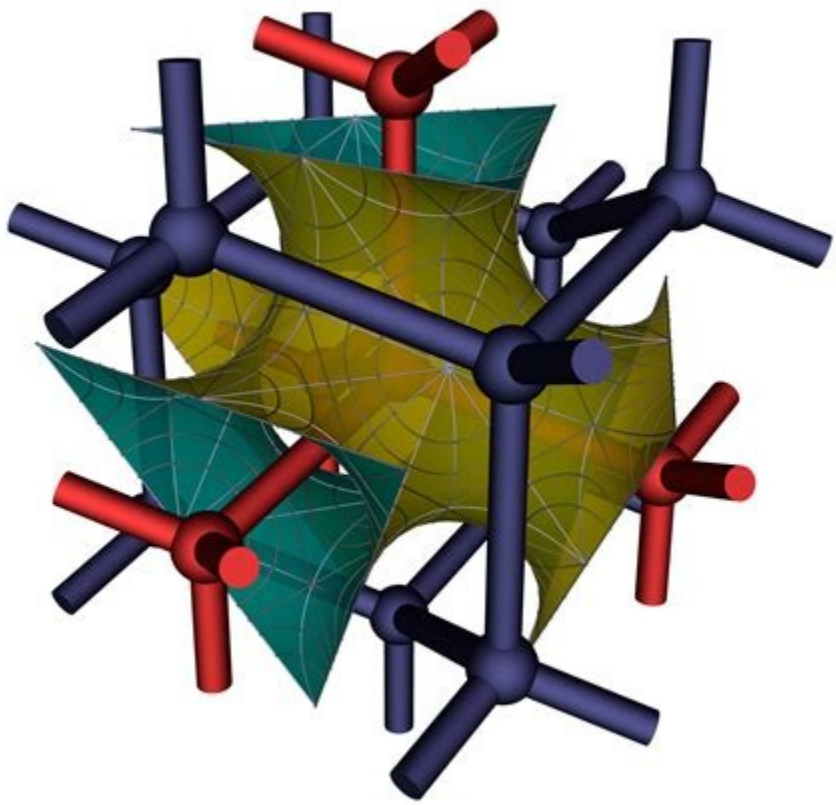

Figure 8.1 — The Schwartz D-Surface: a triply periodic minimal surface with $Fd\bar{3}m$ symmetry, dividing space into two congruent interlocking labyrinths.

Property	Value
Surface Classification	Triply periodic minimal surface (TPMS)
Crystal Symmetry	Cubic — space group $Fd\bar{3}m$ (diamond cubic)
Topology	Divides R^3 into two congruent, interlocking labyrinths
Mean Curvature	$H = 0$ everywhere (definition of a minimal surface)
Gaussian Curvature	$K \leq 0$ everywhere (saddle-like at every point)
Genus per Unit Cell	High genus — complex, multiply-connected topology

8.2 Mapping to Software Architecture

The D-surface's two interlocking labyrinths map directly to the Manifold's communication channels:

D-Surface Element	Software Mapping	Role
Labyrinth A	Write channel	Substrates publish coordinates into this channel
Labyrinth B	Read channel	Substrates consume coordinates from this channel
Surface boundary (A ∩ B)	The Registry	The shared coordinate space — the only point of contact
$Fd\bar{3}m$ symmetry	Substrate equivalence	Every substrate has topologically equivalent access — no privilege

No information crosses from Labyrinth A to Labyrinth B except through the surface boundary. This is the architectural guarantee of decoupling. The cubic symmetry of $Fd\bar{3}m$ ensures that no substrate is "closer" to another in the topology — all substrates are equidistant from the shared surface.

8.3 Why "Diamond"?

The name "Diamond" is not arbitrary. It carries four layers of meaning:

12. **Crystallographic:** The Schwartz D-surface has the same symmetry as the diamond crystal structure ($Fd\bar{3}m$). In a diamond crystal, every carbon atom is bonded to exactly 4 neighbors in a tetrahedral arrangement — maximally symmetric, maximally strong. In the Diamond substrate pattern, every substrate is "bonded" to the shared surface with equal strength.
13. **Clarity:** Diamond is transparent — the architecture is transparent. Every data flow is visible on the surface. There are no hidden channels, no secret callbacks, no buried event handlers.
14. **Hardness:** Diamond is the hardest natural material. The Diamond pattern produces architectures that are robust against coupling, resistant to spaghetti, and structurally sound under extension.
15. **Value:** The architecture itself is the innovation — the intellectual property. The Diamond pattern is the competitive advantage. It is valuable precisely because it is original, principled, and mathematically grounded.

Section 9 — Extending the Manifold

9.1 Adding New Substrates

To add a new substrate to any ButterflyFX application, follow this exact procedure:

16. Create `[name].substrate.js` following the exact class template defined in Section 5.3
17. Register published coordinates in the constructor via `this.registry.registerPublisher()`
18. Implement `sample()` to read needed coordinates from the registry

19. Implement `evaluate()` with domain-specific manifold logic (surface evaluation, gradient computation)
20. Implement `publish()` to write output coordinates to the registry
21. Optionally implement `render()` for visual or audio output
22. Add the substrate instance to the main loop's `substrates` array in `main.js`
23. **No changes needed to any existing substrate** — the Diamond pattern guarantees zero coupling

> **ZERO-COUPLING GUARANTEE**
>
> Step 8 is the critical property. Because substrates communicate only through the Registry surface, adding or removing a substrate has zero impact on any other substrate's code. This is the practical power of the Diamond architecture — the system is extensible by addition, never by modification.

9.2 Adding New Manifold Surfaces

New surfaces can be added to `surface.js` as additional evaluation functions. Any function z = f(x, y) is a valid manifold surface. Each new surface MUST include its gradient function for AI gradient descent.

Surface Name	Equation	Gradient ∇f	Use Case
Paraboloid	$z = x^2 + y^2$	(2x, 2y)	Radial difficulty scaling, distance fields
Wave Field	$z = \sin(x) \cdot \cos(y)$	(cos(x)·cos(y), −sin(x)·sin(y))	Oscillating game states, wave-based audio
Gaussian Bell	$z = e^{-(x^2+y^2)}$	(−2x·z, −2y·z)	Proximity falloff, attention fields, spatial audio
Cubic Relation	$z = x \cdot y^3$	$(y^3, 3xy^2)$	Extreme non-linear response,

Surface Name	Equation	Gradient ∇f	Use Case
			boss battle AI

9.3 Cross-Application Substrate Sharing

Substrates are portable between applications because they depend only on the Registry interface. This is the power of the Diamond architecture — substrates are universal components:

- The **Music Substrate** from Fast Track can run unmodified in Starfighter — it reads game-state coordinates and produces music regardless of what game those coordinates represent
- The **AI Substrate's** gradient-walking algorithm works for any domain — just change the evaluation coordinates. Board game strategy and space combat tactics use the same mathematical framework
- The **Auth Substrate** and **Social Substrate** are application-agnostic — they manage identity and communication for any ButterflyFX title
- The **Commentary Substrate** reads game-state coordinates and generates narrative — the same substrate narrates Fast Track moves and Starfighter battles

---

Section 10 — Quality Standards and Rendering Mandates

10.1 Hi-Poly Rendering Requirements

The visual standard for all ButterflyFX applications is: **"Every frame must be screenshot-worthy."** This is not aspirational — it is a rendering mandate for press kits, store listings, and user experience.

Requirement	Specification
LOD threshold	All objects within 500 units of camera: full hi-poly rendering, no LOD reduction
Close object poly count	Minimum 15,000 triangles per object (pegs, cards, board surface)
Material system	PBR mandatory: albedo, normal, metallic, roughness, ambient occlusion maps
Subsurface scattering	Required for translucent materials (billiard ball pegs, card edges)
Shadows	Real-time shadows with soft penumbra
Post-processing pipeline	Bloom, ambient occlusion, depth of field, anti-aliasing (MSAA 4x minimum)
Frame rate target (1080p, mid-range)	60 fps
Frame rate target (1080p, high-end)	120 fps

10.2 Audio Quality Standards

Requirement	Specification
Sample rate	44,100 Hz minimum
Synthesis engine	Web Audio API with AudioWorklet for custom DSP
Spatial audio	Full 3D sound positioning for all game events
Music loudness	−12 dB LUFS
Effects peak level	−6 dB peak
Reverb	Procedural reverb for environment simulation
Audio files	Zero — every sound synthesized from manifold coordinates

ZERO AUDIO FILES

The Manifold architecture mandates that NO pre-recorded audio files are used in any ButterflyFX application. Every sound — every note, every effect, every ambient tone — is synthesized procedurally from coordinates on the Relation and Form manifold surfaces using the Web Audio API. This is not a cost-saving measure; it is a design principle that ensures infinite variation and mathematically consistent audio.

Section 11 — Commercial Deployment

11.1 Target Platforms

Platform	Build Target	SDK	Submission
Web	kensgames.com	Native browser APIs	Self-hosted
Steam	Electron wrapper	Steamworks SDK	Steam Direct ($100)
Google Play	Capacitor / PWA	Play Console	$25 registration
iOS	Capacitor	Xcode + App Store Connect	$99/year Apple Developer
Xbox	UWP / Electron	ID@Xbox + Xbox Live	ID@Xbox application
PlayStation	Custom	PlayStation Partners SDK	PS Partners application
Nintendo Switch	Custom	Nintendo Dev Portal	Developer application
Meta Quest	WebXR or native	Quest SDK	Quest Store submission

11.2 The Manifold as Competitive Advantage

The Manifold architecture is not merely a technical framework — it is the competitive moat around the Ken's Games ecosystem. Each aspect of the architecture translates to a measurable commercial advantage:

Architecture Feature	Commercial Advantage
Zero audio files	Dramatically smaller download size — faster installs, lower bandwidth costs, competitive app store listings
Procedural content	Infinite variety from minimal assets — every game session sounds and feels unique
Substrate architecture	Easier maintenance, testing, and extension — new features ship faster, bugs isolate cleanly
Mathematical foundation	Reproducible, deterministic behavior — essential for competitive play, replays, and debugging
Golden ratio aesthetics	Universally pleasing visual proportions — the game "feels right" to players without knowing why
Cross-platform substrate portability	Write once, deploy everywhere — substrates are platform-agnostic by design
Original IP	The architecture itself is the innovation — defensible, patentable, and unique in the industry

---

Section 12 — Glossary of Manifold Terms

Term	Definition
Manifold	A mathematical space that locally

Term	Definition
	resembles Euclidean space. In this architecture, the collection of surfaces upon which all computation is performed. The word also refers to the overall system architecture.
Surface	A 2-dimensional manifold embedded in 3D space, defined by a function z = f(x, y). The primary computational primitive. All state, audio, and AI evaluation occurs on surfaces.
Substrate	A self-contained computational module that owns a single domain, publishes output coordinates, and reads input coordinates from shared manifold surfaces. Substrates NEVER call each other directly.
Registry	The Surface Registry — a flat, double-buffered dictionary mapping coordinate names to values. The only shared infrastructure. Contains no logic. Enforces single-writer, multi-reader access.
Coordinate	A named value published to or read from the Registry. The atomic unit of inter-substrate communication. Format: `domain.aspect.detail`.
Gradient (∇f)	The vector of partial derivatives of a surface at a point, indicating the direction of steepest ascent. Used by AI agents for decision-making and optimization.
φ (Phi)	The golden ratio: (1 + √5) / 2 = 1.6180339887... The universal proportion constant governing all spatial and temporal relationships in the Manifold system.
Fibonacci	The integer sequence 1, 1, 2, 3, 5, 8, 13, 21, 34, 55... where each term is the sum of the two preceding terms. Consecutive ratios converge to φ. Governs audio weighting, timing, and scaling.
ButterflyFX	The Dimensional Programming Platform that encompasses the Manifold

Term	Definition
	architecture, the 7-Layer Helix, and the Diamond substrate pattern. The overarching framework within which all Ken's Games titles are built.
Diamond Pattern	The substrate communication pattern named after the Schwartz D-surface. Substrates communicate only through shared surfaces (the Registry), never directly. Guarantees total decoupling and topological equivalence.
Schwartz Surface	A family of triply periodic minimal surfaces (TPMS). The Schwartz D-surface (Diamond surface) provides the topological template for the Diamond communication pattern.
Relation Surface	The primary manifold surface $z = x \cdot y$. A hyperbolic paraboloid used for game state evaluation, basic audio synthesis, and AI decision-making. Gradient: $\nabla f = (y, x)$.
Form Surface	The secondary manifold surface $z = x \cdot y^2$. Used for harmonic-rich audio synthesis, non-linear AI evaluation, and complex decision landscapes. Gradient: $\nabla f = (y^2, 2xy)$.
Spark	Layer 1 of the ButterflyFX 7-Layer Dimensional Helix. The origin point — a single seed value from which all computation unfolds. Mathematical analog: the 0-dimensional point.
Mirror	Layer 2. Duality — every value has its complement. Establishes the read/write separation fundamental to the Diamond pattern. Mathematical analog: 1-dimensional line.
Relation	Layer 3. The first true manifold surface — two independent variables produce a dependent value. WHERE computation happens. Mathematical analog: 2D surface in 3D space.
Form	Layer 4. Structure emerges from relation. Non-linearity and complexity. Hierarchies, layouts, and component trees. Mathematical analog: higher-

Term	Definition
	order curved surfaces.
Life	Layer 5. Time-parameterized curves on surfaces — animation, physics, music. Points move, trajectories unfold, state evolves. Mathematical analog: parameterized curves on manifolds.
Mind	Layer 6. Coherence — the system evaluates its own state. AI gradient descent, pattern recognition, strategic awareness. Mathematical analog: gradient fields and optimization.
Completion	Layer 7. The whole butterfly. All substrates unified, all surfaces active, all dimensions carried. Completion becomes the Spark of the next spiral. Mathematical analog: the full manifold.
Potential	First of three point states. Unaddressed possibility — the coordinate exists in mathematical space but has not been evaluated. Latent capability.
Null	Second of three point states. Addressed but valueless — the coordinate has been visited but carries no manifest value. Pure location in the topology.
Manifest	Third of three point states. Addressed and valued — the coordinate carries a concrete value (note, position, decision, color). Actualized state.
Saddle Point	A critical point on a surface where the gradient is zero but the point is neither a maximum nor a minimum. The origin of the Relation surface $z = x \cdot y$ is a saddle point.
Hyperbolic Paraboloid	The geometric shape of the Relation surface $z = x \cdot y$. A doubly-ruled saddle surface that curves upward in one direction and downward in the perpendicular direction.
Labyrinth	One of the two interlocking channels created when the Schwartz D-surface divides R^3. Labyrinth A = write channel; Labyrinth B = read channel.
$Fd\bar{3}m$	The space group (symmetry group) of

Term	Definition
	the diamond cubic crystal structure and the Schwartz D-surface. Ensures every substrate has topologically equivalent access to the Registry.
Single-Writer	Registry constraint: only one substrate MAY publish to a given coordinate. Prevents write conflicts and ensures deterministic state.
Multi-Reader	Registry property: any number of substrates MAY read any coordinate. Information flows freely from surfaces to observers.
Double-Buffer	The Registry maintains two buffers (front and back). Reads come from the front buffer; writes go to the back buffer. On swap, they exchange — ensuring frame consistency.
Surface Evaluation	The act of computing $z = f(x, y)$ for a given (x, y) coordinate on a manifold surface. The fundamental computational operation of the Manifold architecture.
Gradient Descent	An optimization algorithm that moves in the direction opposite the gradient ($-\nabla f$) to find minima, or along the gradient ($+\nabla f$) to find maxima. Used by AI substrates for decision-making.
Step Sequencer	A music composition tool with 16 slots per bar. Each slot triggers sounds derived from manifold coordinates. The Music Substrate's primary rhythm engine.
PBR	Physically Based Rendering — a shading model using albedo, normal, metallic, roughness, and ambient occlusion maps to simulate real-world material behavior under lighting.
SSS	Subsurface Scattering — a rendering technique for translucent materials (skin, wax, pegs) where light penetrates the surface and scatters before exiting.
LOD	Level of Detail — the practice of reducing polygon count for distant

Term	Definition
	objects. In the Manifold system, LOD reduction is prohibited within 500 units of the camera.

> *"This document is not a description of what might be built. It is the blueprint of what SHALL be built. Every surface, every substrate, every golden-ratio cascade documented here is a binding specification. The Manifold is the foundation. The Diamond pattern is the architecture. ButterflyFX is the framework. And φ is the constant that binds them all."*
>
> — Ken Bingham, Architect — ButterflyFX / Ken's Games

Ken Bingham

Creator & Architect — Manifold / ButterflyFX / Ken's Games

April 12, 2026

MANIFOLD — Diamond Substrate Architecture Design Document (Diamond Edition) v1.0

© 2026 Ken Bingham / Ken's Games. All rights reserved. Proprietary and confidential.